CS401 Intelligent Robotics Lab 05 Report

Date: 2026-04-01

This report summarizes the setup and validation of the `limo` vehicle simulation in ROS Noetic, including RViz/Gazebo workflow, control methods, and the lane-detection demo pipeline.

1. Objectives

1. Build the `limo_ws` workspace and compile the simulation packages.
2. Display the LIMO model in RViz and start Ackerman-mode Gazebo simulation.
3. Review control methods such as `rqt_robot_steering` and keyboard teleoperation.
4. Run the camera projection, lane detection, and lane-following control pipeline.

2. Environment

Remote host: `ubuntu@192.168.40.128`
Remote OS: Ubuntu 20.04
ROS distro: `noetic`
Workspace: `/home/ubuntu/limo_ws`
Repository: `ugv_gazebo_sim`

```
mkdir -p ~/limo_ws/src
cd ~/limo_ws/src
git clone https://github.com/Intelligent-Robot-Course/ugv_gazebo_sim.git
cd ~/limo_ws
rosdep install --from-paths src --ignore-src -r -y
catkin_make
```

The workspace was built successfully on the remote Ubuntu machine before the runtime checks below were performed.

3. Launch Validation

```
ROS_DISTRO=noetic
roslaunch --files limo_description display_models.launch
roslaunch --files limo_gazebo_sim limo_ackerman.launch
roslaunch --files limo_gazebo_sim limo_demo.launch
roslaunch --files limo_camera extrinsic_camera_calibration.launch
roslaunch --files limo_detect detect_lane.launch
roslaunch --files limo_driving turtlebot3_autorace_control_lane.launch
```

All required launch files resolved correctly during this session. The detailed node/topic summary is saved in `lab5_validation_summary.txt`.

4. RViz and Gazebo Workflow

The reference workflow from the supplied lab material is shown below. These figures document the expected RViz model display and Ackerman-mode Gazebo scene. In parallel, the live session verified that Gazebo published `/cmd_vel`, `/limo/scan`, and `/limo/color/image_raw`, with RViz subscribing to the sensor topics.

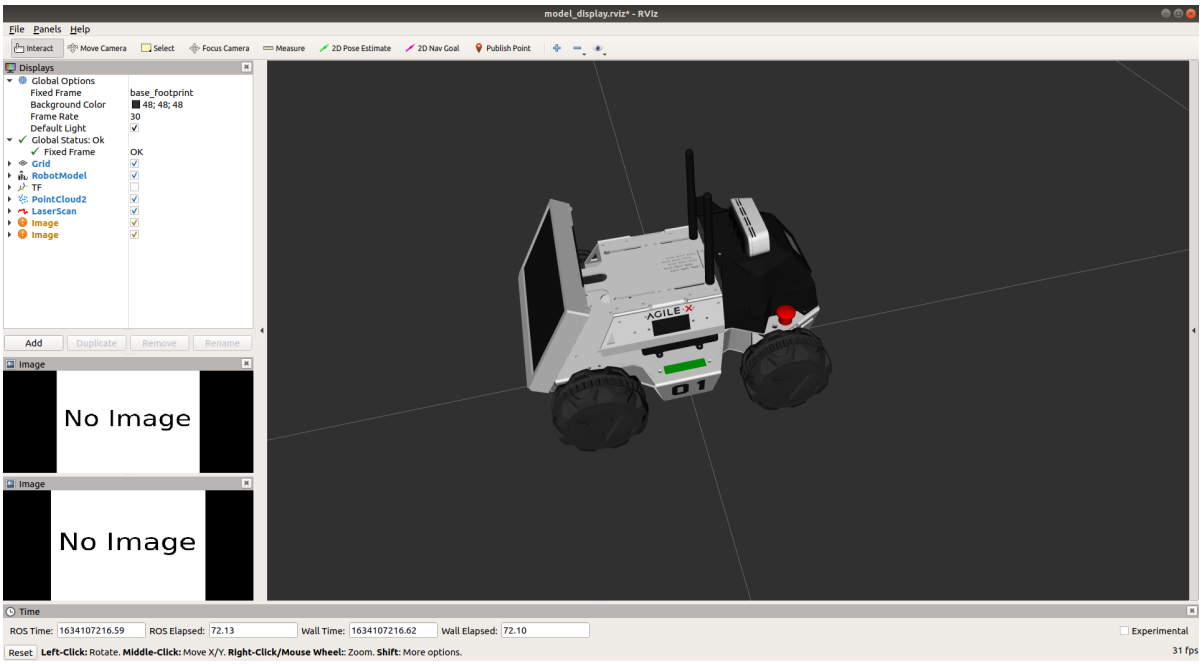

Figure 1. Reference RViz model-display result from the provided Lab 5 material.

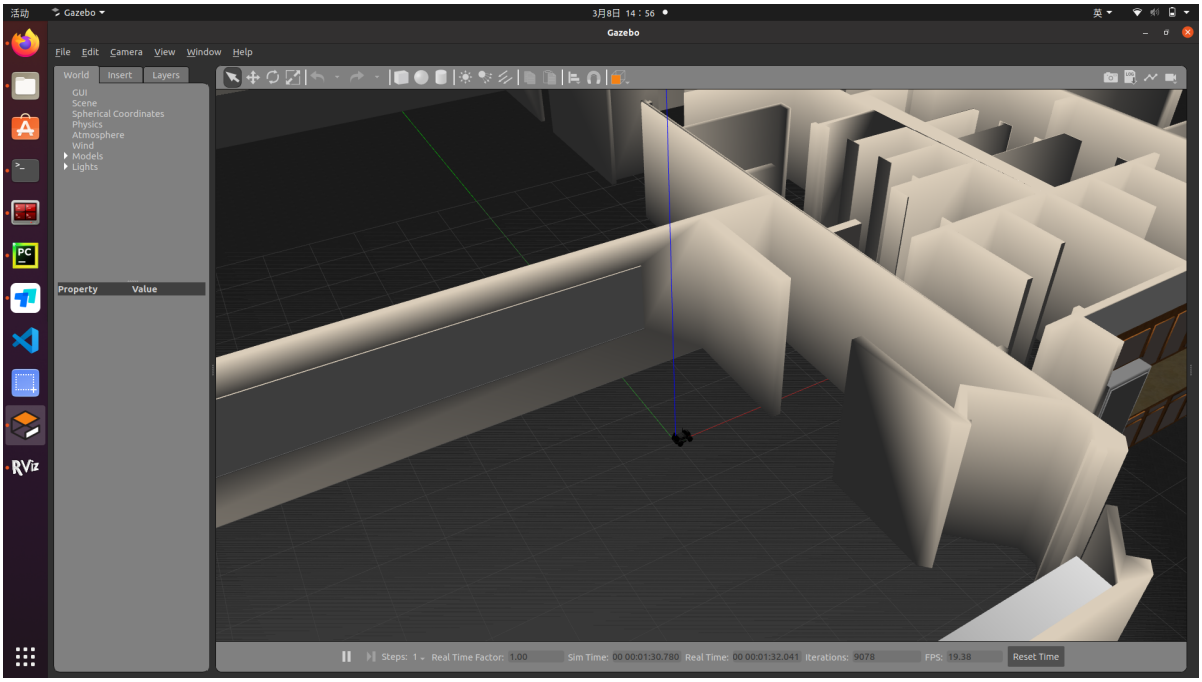

Figure 2. Reference Ackerman-mode Gazebo result from the provided Lab 5 material.

```
/controller_spawner
/gazebo
/gazebo_gui
/robot_state_publisher
/rviz

/clock
/cmd_vel
/limo/color/image_raw
/limo/depth/image_raw
/limo/imu
/limo/scan
/tf
```

5. Control Methods

The lab introduces two basic ways to move the robot: the GUI-based `rqt_robot_steering` panel and keyboard teleoperation. The figures below are reference screenshots from the supplied lab material.

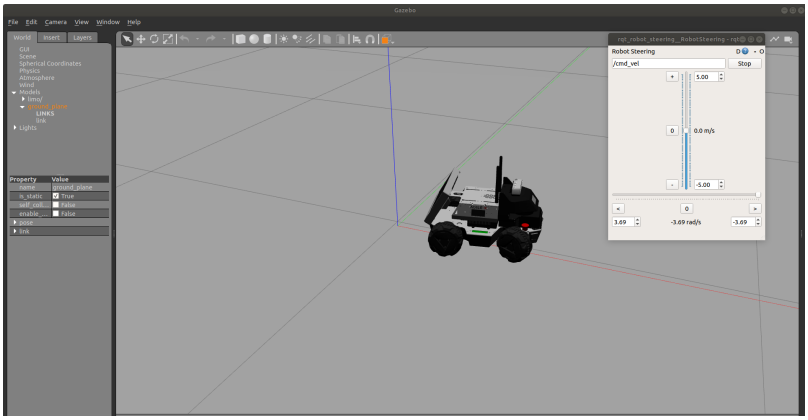

Figure 3. Reference rqt_robot_steering interface.

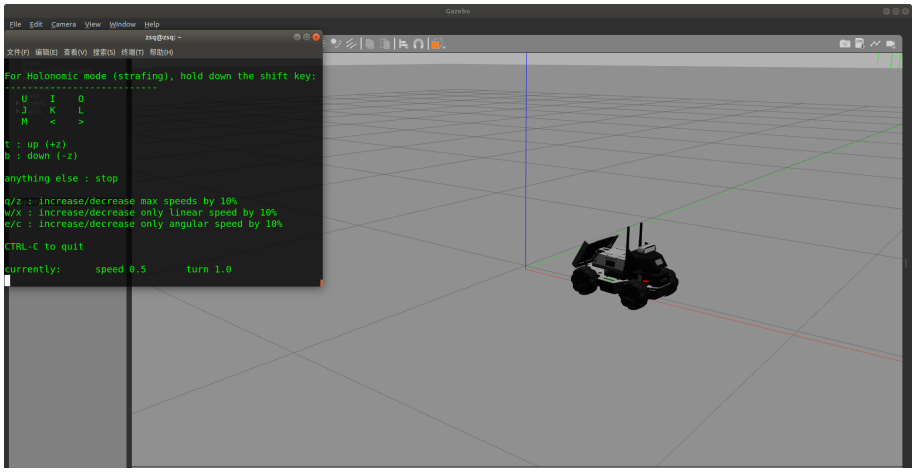

Figure 4. Reference keyboard-control result.

6. Lane-Detection Demo

For the autorace-style demo, the race world was started with `limo_demo.launch`, then the camera projection, lane detection, and lane-following controller were launched on top of the Gazebo simulation. The live topic chain was verified as `/camera/image_projected_compensated -> /detect/image_lane/compressed -> /detect/lane -> /cmd_vel`.

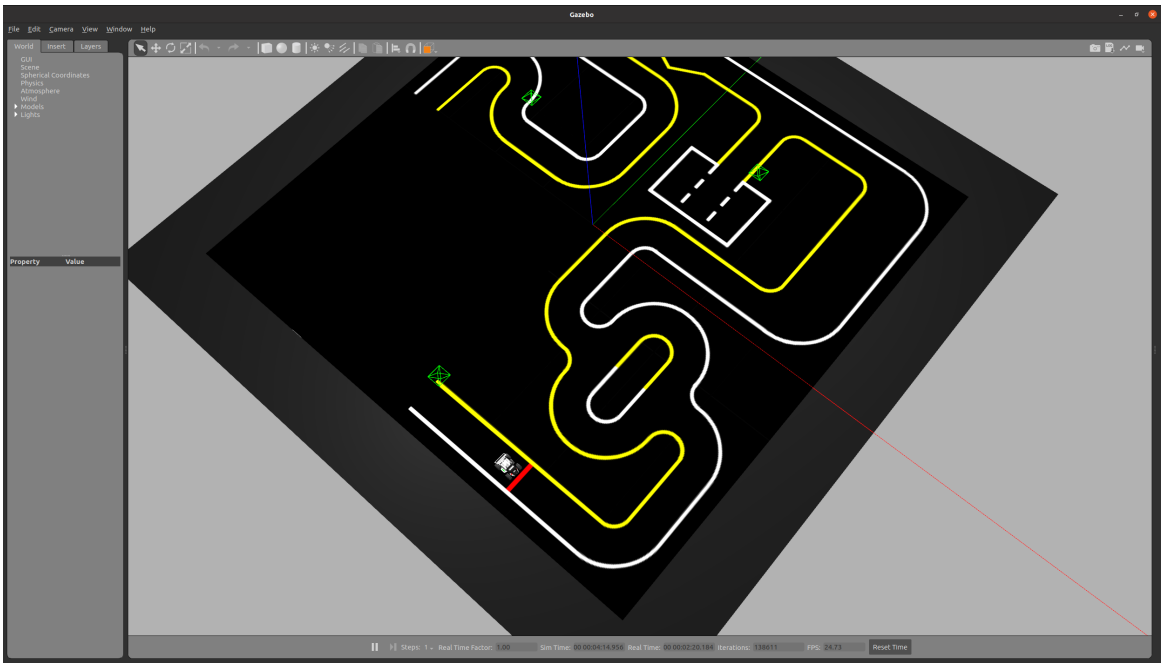

Figure 5. Reference demo world from the provided Lab 5 material.

Figure 6. Live projected camera image saved from /camera/image_projected_compensated.

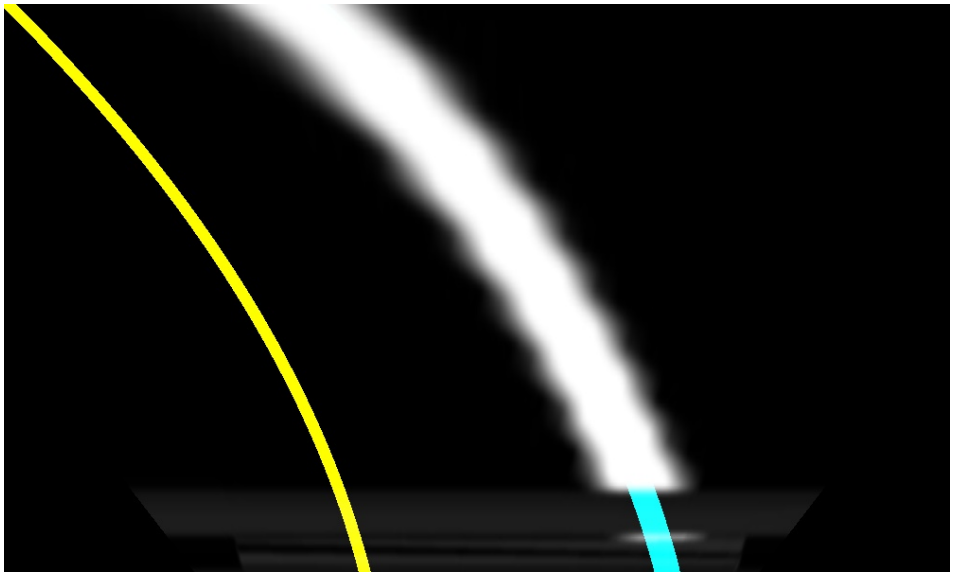

Figure 7. Live lane-detection result saved from /detect/image_lane/compressed.

```
Type: std_msgs/Float64
Topic: /detect/lane
Publisher: /detect_lane
Subscriber: /control_lane

Type: geometry_msgs/Twist
Topic: /cmd_vel
Publisher: /control_lane
Subscriber: /gazebo

sample /detect/lane value: -163.10505252446842
```

7. Runtime Fix

During the first live run of `detect_lane.launch`, the original node crashed with `AttributeError: 'DetectLane' object has no attribute 'mov_avg_left'`. A minimal fix was applied by initializing the lane-fit state and moving-average arrays, and by guarding the averaging step when the history buffers are still empty. After this fix, the lane-detection and control nodes ran without the previous callback error.

```
Patched file:
/home/ubuntu/limo_ws/src/ugv_gazebo_sim/limo/limo_detect/nodes/detect_lane

Saved local copy:
LAB5/generated/detect_lane_fixed.py
```

8. Conclusion

Lab 5 was reproduced successfully on the remote Ubuntu machine. The workspace was built, the LIMO simulation launch chain was validated, and the camera-projection and lane-detection pipeline was brought up end-to-end. The final report combines the provided workflow references with live validation evidence and live ROS-generated result images from the current session.